

DAY 3 — Special Permissions Deep Dive + Access Control Lists (ACL)

Learning Objectives

By the end of today, you will:

- Master SUID, SGID, and Sticky Bit — not just definitions, but when/why they're used in production
- Understand why standard owner/group/others permissions aren't always enough
- Use ACLs (`setfacl`/`getfacl`) to grant fine-grained access — a real enterprise requirement
- Handle multi-team file-sharing scenarios the way a real sysadmin would

Daily Outcome

You can explain and demonstrate special permissions AND solve a permission problem that standard `chmod` cannot solve — using ACLs instead.

1. Why This Matters (WHY before HOW)

Production reality: Standard permissions (owner/group/others) only let you define access for ONE user and ONE group. But real companies have situations like:

"The `/shared/reports` folder needs: `alice` (finance) = read/write, `bob` (audit) = read-only, everyone else = no access."

Standard `chmod` **cannot do this** — it only handles one owner + one group. This is exactly where ACLs come in, and exactly why they're asked about in intermediate-level interviews (shows you've gone beyond the basics).

2. Special Permissions — Deep Dive

SUID (Set User ID) — Symbol: `s` in owner's execute slot

Beginner: When a program has SUID set, it runs with the **file owner's** privileges — not the privileges of the person running it.

Real production example: The `passwd` command. When a regular user runs `passwd` to change their own password, it needs to write to `/etc/shadow`, which is root-only. `passwd` has SUID set and is owned by root — so it temporarily runs as root, letting a normal user update their password safely, without giving them root access generally.

```
bash

ls -la /usr/bin/passwd
-rwsr-xr-x 1 root root ... /usr/bin/passwd
```

Notice the `(s)` where you'd normally see `(x)` in the owner slot.

Setting SUID:

```
bash

chmod u+s myprogram
# or numerically:
chmod 4755 myprogram
```

The `(4)` prefix = SUID.

Security consideration (important for interviews): SUID on the wrong binary is a massive security risk — if a SUID root program has a vulnerability, an attacker can exploit it to gain root access. Real security audits regularly scan for unnecessary SUID binaries with:

```
bash

find / -perm -4000 -type f 2>/dev/null
```

SGID (Set Group ID) — Symbol: `(s)` in group's execute slot

Beginner: On a directory, SGID makes all new files/folders created inside automatically inherit the directory's group — not the creating user's default group.

Real production example: A `/shared/dev-team` folder used by 5 developers. Without SGID, every new file takes the creator's personal default group, so teammates might not have access. With SGID set, every file automatically belongs to `dev-team` group, so everyone can access it seamlessly.

```
bash

chmod g+s /shared/dev-team
# or:
chmod 2775 /shared/dev-team
```

The `(2)` prefix = SGID.

Sticky Bit — Symbol: `(t)` in others' execute slot

Beginner: On a shared directory, the sticky bit ensures users can only delete/rename **their own** files, even if they have write access to the directory.

Real production example: `/tmp` — hundreds of users/processes write files here. Without the sticky bit, any user could delete any other user's temp files. With it, `user_a` can never delete `user_b`'s file, even though both can write to `/tmp`.

```
bash
ls -ld /tmp
drwxrwxrwt 12 root root 4096 Jul 5 10:00 /tmp
```

Notice the `(t)` at the end.

```
bash
chmod +t /shared/dropzone
# or:
chmod 1777 /shared/dropzone
```

The `(1)` prefix = sticky bit.

Quick Reference Table

Special Perm	Numeric Prefix	Symbol Position	Common Use
SUID	4	<code>(s)</code> in owner's x	<code>passwd</code> , <code>sudo</code> , <code>ping</code>
SGID	2	<code>(s)</code> in group's x	Shared team directories
Sticky Bit	1	<code>(t)</code> in others' x	<code>/tmp</code> , shared upload folders

3. Access Control Lists (ACL) — The Advanced Solution

Why ACLs exist: Standard Linux permissions only support ONE owner and ONE group per file. ACLs let you assign permissions to **multiple specific users and groups** on the same file — no workarounds needed.

Real production example (revisit the earlier scenario):

`/shared/reports`: `alice` needs read/write, `bob` needs read-only, default owner/group get nothing.

This is a textbook ACL use case, and it comes up in real enterprise environments (finance, healthcare, government) where compliance requires exact, auditable access per user.

Commands

```
bash
sudo setfacl -m u:alice:rw /shared/reports/file.txt
```

→ `setfacl` = set ACL. `-m` = modify. `u:alice:rw` = grant user `alice` read+write.

```
bash
sudo setfacl -m u:bob:r /shared/reports/file.txt
```

→ Grants `bob` read-only access.

```
bash
getfacl /shared/reports/file.txt
```

→ Shows ALL ACL entries on the file — this is your verification command.

Expected output:

```
# file: file.txt
# owner: root
# group: root
user::rw-
user:alice:rw-
user:bob:r--
group::r--
mask::rw-
other::---
```

```
bash
sudo setfacl -x u:bob /shared/reports/file.txt
```

→ `-x` = remove a specific ACL entry (removes bob's access here).

```
bash
```

```
sudo setfacl -b /shared/reports/file.txt
```

→ `-b` = remove ALL ACL entries, back to standard permissions only.

Important detail interviewers look for: After setting an ACL, `ls -la` will show a `+` at the end of the permission string:

```
-rw-rw----+ 1 root root 0 Jul 5 file.txt
```

That `+` tells you "this file has extra ACL rules beyond standard permissions" — a detail many candidates miss.

4. Cheat Sheet

Command	Purpose
<code>chmod u+s <file></code>	Set SUID
<code>chmod g+s <dir></code>	Set SGID
<code>chmod +t <dir></code>	Set sticky bit
<code>find / -perm -4000</code>	Find all SUID binaries (security audit)
<code>setfacl -m u:<user>:<perm> <file></code>	Grant ACL to a user
<code>setfacl -m g:<group>:<perm> <file></code>	Grant ACL to a group
<code>getfacl <file></code>	View ACL entries
<code>setfacl -x u:<user> <file></code>	Remove one ACL entry
<code>setfacl -b <file></code>	Remove all ACL entries

5. Real World Scenario

Scenario: Your company's `/shared/finance` folder needs three different access levels: the finance team (read/write), the audit team (read-only), and a specific external contractor (read-only, but ONLY for one file, not the whole folder). Standard groups can't cleanly express "one specific person gets access to one specific file."

Solution walkthrough:

1. `sudo setfacl -m g:finance:rw /shared/finance` — finance team gets RW on the folder
2. `sudo setfacl -m g:audit:r /shared/finance` — audit team gets read-only
3. `sudo setfacl -m u:contractor_john:r /shared/finance/report_q1.txt` — ONLY this file, ONLY this user
4. `getfacl /shared/finance/report_q1.txt` — verify before closing the ticket

This kind of layered access is exactly what's tested in "design an access control scenario" interview questions.

6. Hands-On Lab

Steps:

1. Create a test directory: `mkdir ~/labday3 && cd ~/labday3`
2. Create a file: `touch report.txt`
3. Check default permissions: `ls -la report.txt`
4. Grant ACL access to a user (use `testuser` from Day 2): `sudo setfacl -m u:testuser:rw report.txt`
5. Verify: `getfacl report.txt`
6. Confirm the `(+)` appears: `ls -la report.txt`
7. Create a shared directory: `mkdir teamshare`
8. Set SGID on it: `chmod g+s teamshare`
9. Verify: `ls -ld teamshare` (look for `(s)` in group execute position)
10. Remove the ACL entry: `sudo setfacl -x u:testuser report.txt`
11. Verify removal: `getfacl report.txt`

Expected Output (step 6):

```
# file: report.txt
# owner: anuj
# group: anuj
user::rw-
user:testuser:rw-
group::r--
mask::rw-
other::r--
```

Common Errors:

Error	Cause	Fix
<code>setfacl: command not found</code>	ACL package not installed	<code>sudo apt install acl</code> (Ubuntu) or <code>sudo yum install acl</code> (RHEL)
ACL not taking effect	Filesystem not mounted with ACL support (rare on modern systems)	Check `mount`
<code>+</code> not showing in <code>ls -la</code>	ACL was removed or never applied	Re-run <code>getfacl</code> to confirm current state

7. Mini Assignment

1. Set up a scenario: one file, three users, three DIFFERENT permission levels, using ACLs only (not standard chmod).
2. Run `getfacl` and paste the output showing all three access levels.
3. Explain in 2-3 sentences why you couldn't achieve this with standard `chmod` alone.

8. Quiz (10 Questions)

1. What does SUID do differently from normal execute permission?
2. Give a real command that uses SUID by default.
3. What problem does SGID solve for team folders?
4. Why does `/tmp` have the sticky bit set?
5. What numeric prefix represents SGID?
6. What does the `+` at the end of `ls -la` output indicate?

7. What command view all ACL entries on a file?
 8. How do you remove ALL ACL entries from a file?
 9. Why can't standard Linux permissions solve the 3-team-different-access scenario?
 10. What security risk does SUID introduce if misused?
-

9. Interview Preparation

Q1: What's the difference between SUID and SGID?

| Expected: SUID = runs as file owner; SGID = new files inherit directory's group.

Q2 (Scenario): You need to give one contractor read-only access to a single file, without changing anything else. How do you do it?

| Expected: `setfacl -m u:username:r filename` — NOT changing group/chmod.

Q3 (Cross-question): "What if the filesystem doesn't support ACLs?"

| Expected: Mention checking filesystem type (ext4/xfs support ACLs natively, but need the `acl` mount option in some setups); alternative would be restructuring with more specific groups.

Q4: Why is SUID considered a security risk?

| Expected: If a SUID root binary has a vulnerability (e.g., buffer overflow), an attacker can exploit it to escalate to root privileges — this is why security teams audit SUID binaries regularly.

10. English Communication

10 Technical Words: inherit, escalation (as in privilege escalation), granular, entry, mask, audit, exploit, vulnerability, compliance, entitlement

10 Business Words: stakeholder sign-off, contractor, confidentiality, data governance, access review, segregation of duties, liability, disclosure, safeguard, accountability

Speaking Task: Explain out loud in under 90 seconds: "What's the difference between standard Linux permissions and ACLs, and give one real scenario where you'd need ACLs." Focus on pronouncing "granular" and "entitlement" clearly.

Writing Task: Write a short email (4-5 sentences) to a "team lead" explaining that you've set up ACL-based access for a new contractor, specifying exactly what access they have. Professional tone. I'll correct it.

Daily Summary

- Mastered SUID, SGID, and Sticky Bit with real production reasoning
- Learned ACLs for scenarios standard permissions can't handle
- Practiced a layered, multi-team access control scenario
- Learned the security implications of special permissions

Revision Checklist

- ☐ Can explain SUID/SGID/Sticky Bit without notes
- ☐ Can set and verify an ACL entry from memory
- ☐ Completed the hands-on lab
- ☐ Attempted all 10 quiz questions

Homework

- Finish the Mini Assignment
- Run `find / -perm -4000 2>/dev/null` on your system and identify 3 SUID binaries you find — look up what each one does
- Submit your email writing task

Tomorrow Preview

Day 4: File Operations — find, grep, tar, and compression — the everyday commands that make you fast and effective as an admin, plus real log-searching scenarios.

Motivational Note

ACLs are something a lot of "Linux certified" people can't actually explain hands-on. You're already ahead by doing this practically. Keep the momentum. 🗝️🚀

Before Day 4: please paste your actual `getfacl` output from the lab and your email writing task — even 3-4 lines is enough. It's how I make sure Day 3 actually stuck before we build on it.